MORDERN APPLICATION DEVELOPMENT-1

APP DEVELOPMENT PROJECT REPORT

By
NAME: MRIDUL GOYAL
Roll No.: 24F2003142
Email.:24f2003142@ds.study.iitm.ac.in

INDIAN INSTITUTE OF TECHNOLOGY, Madras
Chennai-711103, Tamil Nadu, India
Date: 29/11/2025

About Me

I am an undergraduate engineering student with a strong interest in software development and real-world application building. I enjoy designing full-stack systems, especially those involving database management, user interfaces, and workflow automation. This project allowed me to strengthen my understanding of Flask, SQLAlchemy, and practical hospital workflow modeling through hands-on development.

---

1. Project Details

Project Title: Lifeline Hospital Management System (HMS)

Problem Statement:

Hospitals require efficient and centralized systems to manage patients, doctors, appointments, departments, and treatment records. However, many healthcare facilities still rely on manual registers, paper-based documentation, or disconnected software tools. This results in several challenges such as:

1. Difficulty in maintaining accurate and up-to-date records
2. Overlapping or conflicting appointment schedules
3. Slow patient registration and approval workflows
4. Lack of centralized access to patient treatment history
5. Inefficient communication between hospital staff and departments

To address these gaps, a Hospital Management System (HMS) web application is required. The system must digitize hospital operations, streamline record-keeping, and enable Admins, Doctors, and Patients to interact with the platform based on clearly defined roles. The goal is to improve data accuracy, reduce manual workload, and enhance overall hospital workflow efficiency.

Approach:

The Lifeline Hospital Management System was developed as a modular Flask-based web application, focusing on role-based access and streamlined hospital workflows. The application follows a clean separation of concerns between backend logic, database models, and frontend templates.

The backend is implemented using Flask combined with SQLite/SQLAlchemy for database management. Each hospital entity—such as users, doctors, patients, departments, appointments, and patient visits—is represented as a database model or table. The application routes handle role-specific functionalities such as doctor approvals, appointment booking, patient registration, and updating treatment records.

The frontend uses Jinja2 templates and Bootstrap to provide a responsive and user-friendly interface for Admins, Doctors, and Patients. Dynamic pages are rendered using data fetched from the database, ensuring real-time updates across dashboards.

The system implements:

1. Role-based access flow for Admin, Doctor, and Patient
2. Doctor approval queue using a pending request table (que)
3. Admin CRUD operations for departments, doctors, and patients
4. Avoidance of record conflicts by storing appointment status (Booked/Completed/Cancelled)
5. Patient visit recording system with prescriptions, notes, tests done, and diagnosis
6. SQLite foreign key enforcement to maintain data consistency

Overall, the approach focuses on building a structured and scalable HMS with clear workflows, minimal dependencies, and easy deployment for small to medium-sized healthcare facilities.

---

2. AI/LLM Declaration

I used ChatGPT (GPT-5.1) during the development of this project primarily for assistance in the following areas:

- Improving code readability and restructuring certain route functions
- Writing boilerplate SQLAlchemy model snippets
- Generating documentation-friendly descriptions for APIs and workflows
- Helping with HTML/Jinja template formatting suggestions
- Clarifying error messages and debugging approaches

The total extent of AI usage is approximately **10–15%**, limited to guidance, explanation, and improving code style.

All major components — including system design, database structure, route logic, CRUD operations, and integration — were implemented manually by me.
No AI-generated code was directly copied without validation or modification.

I confirm that the entire project logic, testing, debugging, and final implementation have been completed independently.

---

3. Technologies and Frameworks Used

Technology / Library	Purpose / Role in Project
Flask	Core backend framework used for routing, request handling, and application logic.
SQLite	Lightweight relational database used to store users, doctors, patients, appointments, departments, and visit records.
SQLAlchemy	ORM used to map Python classes to database tables and simplify CRUD operations.
Jinja2	Template engine for rendering dynamic HTML pages based on user roles and backend data.
Bootstrap 5	Used for responsive UI design, layout structure, modals, tables, and easy styling.
HTML5 & CSS3	Base technologies for frontend page structure and custom styling.
JavaScript (vanilla)	Used for dynamic UI behavior such as modals, adding/removing prescription rows, and form interactions.
SQLite Foreign Keys	Ensures relational integrity between patients, doctors, appointments, and visits.

---

4. Database Schema / ER Diagram

Tables:

1. **Users:** This table stores the login credentials and basic identity information of every system user. It is the central authentication table that differentiates between Admin, Doctor, and Patient roles.

2. **Doctors:** Contains professional and departmental details of doctors. It extends the users table by storing specialization, biography, and department information required for hospital operations.

3. **Patients:** Stores personal and demographic information of registered patients. It links to the users table and enables the system to maintain medical profiles for each patient.

4. **Departments:** Represents the hospital's departments (e.g., Cardiology, Neurology). It is used to categorize doctors and appointments and makes navigation structured for users.

5. **Appointments:** Tracks booking details between patients and doctors. It stores appointment status (Booked, Completed, Cancelled) and helps prevent scheduling

conflicts.

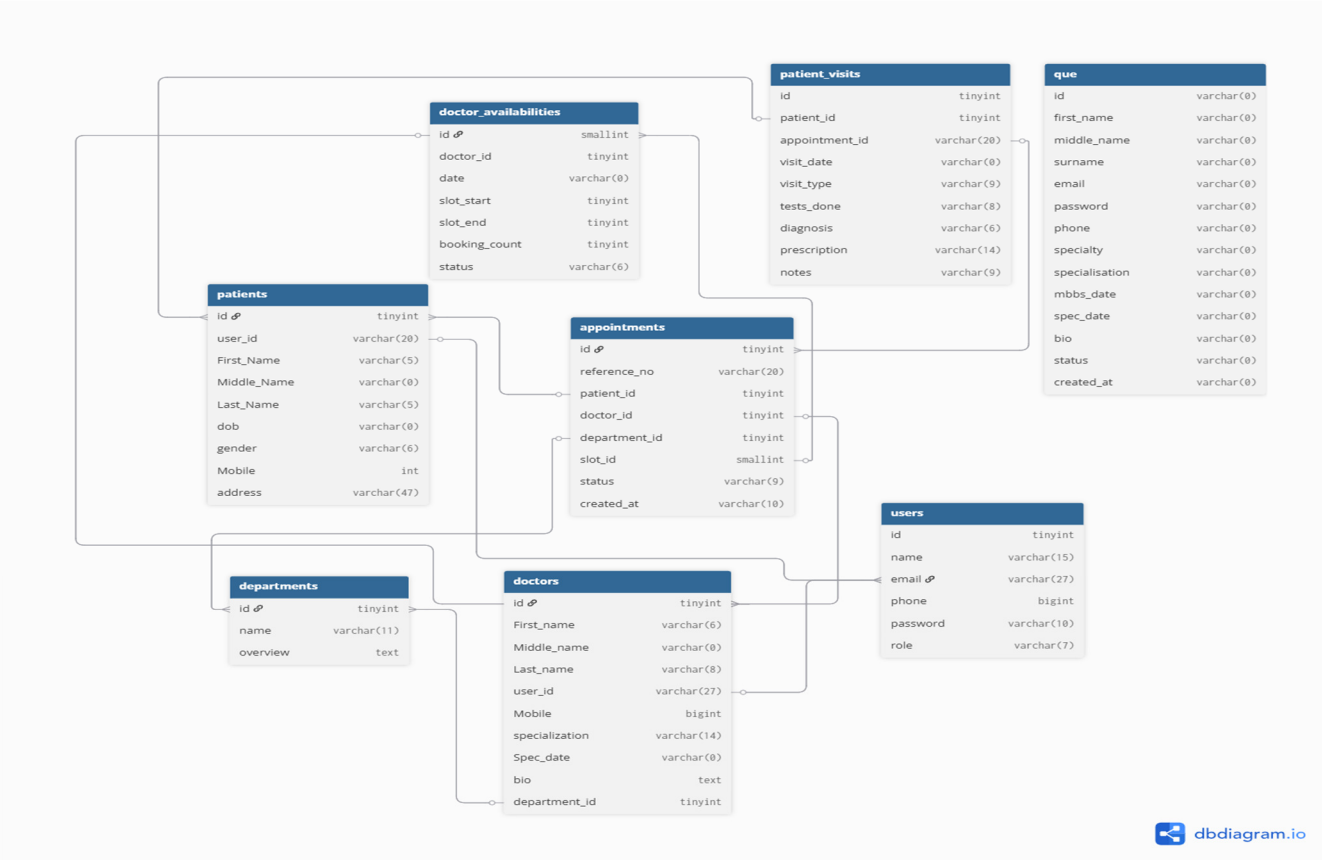
6. **patient_visits:** Stores medical visit records created by doctors after a consultation. It includes diagnosis, prescriptions, tests done, and notes, forming the patient's treatment history.

7. **Que:** A temporary table used for managing doctor registration requests. All new doctors are first added to this queue and appear for admin approval before being officially registered.

8. **doctor_availabilities:** Holds availability schedules for doctors. It helps manage time slots and restricts bookings based on the doctor's working hours.

Relationships:

- One-to-One → `users → doctors, users → patients, appointments → patient_visits`
- One-to-Many → `departments → doctors, patients → appointments, doctors → appointments, doctors → doctor_availabilities,`
- **`que`** `(standalone workflow table)`



5. API Resource Endpoints

Endpoint	Method(s)	Purpose / Key inputs
`/`	GET	Public home page (renders index.html).
`/about`	GET	Public about page.
`/contact`	GET	Public contact page.
`/login`	GET, POST	Login page / authenticate form (input_username, input_password). POST delegates to authenticate() which redirects by role.
`/Doctor_Register`	GET, POST	Doctor signup form: on POST collects doctor fields and inserts into que (pending approval).
`/patient_register`	GET, POST	Patient signup: creates users + patients rows from form fields.
`/doctor/<user_name>`	GET	Doctor dashboard — shows doctor info and booked appointments for that doctor (uses user_name to find doctor).
`/doctor/slot/<user_name>`	GET	Generates/returns next 7 days of doctor availability slots and renders slot management page. Ensures slots exist for the period.
`/doctor/<user_name>/slots/toggle`	POST (JSON)	Toggle a slot open/closed. Expects JSON { "slot_id": <id> }. Returns JSON with new status or error (uses doctor_user context).
`/doctor/<user_name>/slots/book`	POST (JSON)	Increment booking count for a slot (used to simulate a booking via UI). Expects JSON { "slot_id": <id> }. Returns JSON with bookings_count or error.
`/doctor/patient_visit/<user_name>`	GET,	View patient visit history

Endpoint	Method(s)	Purpose / Key inputs
	POST	page for a doctor (fetches patient_visits for given patient_id from form).
/doctor/<user_name>/visit_update/<option>	GET, POST	Multi-purpose doctor action:
/doctor/<user_name>/change_password	POST	Updates patient personal details
/patient/<user_name>	GET	Patient dashboard
/patient/patient-histoy/<user_name>	GET	View patient visit history
/patient/department-details/<user_name>	POST	View doctors by department
/patient/doctor-details/<user_name>	POST	Show selected doctor details
/patient/booking_details/<user_name>	POST	Load booking page for a doctor
/patient/appointments/confirm/<user_name>	POST	Confirm and create appointment
/patient/cancel/<user_name>	POST	Cancel an appointment
/patient/update-profile/<user_name>	POST	Update patient profile
/patient/change_password/<user_name>	POST	Change patient password

---

---

6. Video Presentation

Drive Link:
https://drive.google.com/file/d/1PYAbaKmgBL2jXDPpnsJXLZZpuDiS4kO0/view?usp=sharing
(Accessible to all with "View" permission.)